Quantum Network-Based Prediction of Cancer Driver Genes

Patrícia Marques,[1] Andreas Wichert,[1,2] and Bruno Coutinho[3,4]

[1] *Instituto Superior Técnico, University of Lisbon, Lisboa, Portugal*
[2] *GAIPS, INESC-ID, Porto Salvo, Portugal*
[3] *Instituto de Telecomunicações, Lisboa, Portugal*
[4] *German Aerospace Center (DLR), Cologne, Germany*

(Dated: August 16, 2025)

Identification of cancer driver genes is fundamental for the development of targeted therapeutic interventions. The integration of mutational profiles with protein-protein interaction (PPI) networks offers a promising avenue for their detection [1, 2], but scaling to large networks is computationally demanding. Quantum computing offers compact representations and potential complexity reductions, and while it has been applied to other network-based biological inference tasks, including link prediction, it has not yet been used for driver gene prediction. We present a supervised quantum framework that combines mutation scores with network topology via a novel state preparation scheme, *Quantum Multi-order Moment Embedding* (QMME), motivated by the classical method of Gumpinger et al. [3]. QMME encodes low-order statistical moments over the mutation scores of a node's immediate and second-order neighbors, and encodes this information into quantum states. These are used as inputs to a kernel-based quantum binary classifier that discriminates known driver genes from others. Simulations on a real PPI network demonstrate competitive performance relative to classical baselines. A brief complexity analysis suggests the approach could achieve a quantum speedup in network-based cancer gene prediction. This work underscores the potential of supervised quantum graph learning frameworks to advance biological discovery.

I. INTRODUCTION

A central objective in oncology is to identify and catalogue genes underlying human cancer [4–6]. While genes with high mutation rates are likely drivers, most *cancer driver genes* have few mutations. In these cases, frequency-based methods alone cannot reliably distinguish rare cancer driver genes from irrelevant ones [5]. One explanation lies in the networked nature of cellular processes: genes operate within pathways and complexes, so cancer may result from disruption at the pathway level rather than mutations in specific individual genes. Recent research has embraced this interaction-based perspective of cancer, integrating biological networks with statistical measures of gene-cancer associations to identify novel candidate cancer driver genes [1, 7–9]. In these networks, nodes correspond to genes and edges represent relationships between them. Protein-protein interaction (PPI) networks [10–12] are a widely used representation of gene interactions, as they integrate information from diverse data sources, tissues, and molecular processes.

Previous studies have combined PPI networks with gene-level cancer association scores, using clustering or network propagation methods. A common choice for these gene scores is the MutSig P-value [13, 14], which estimates the significance of observed mutations by considering (i) the overall mutational burden, (ii) clustering of mutations within a gene's coding sequence, and (iii) the functional impact of mutations. NetSig, for instance, focuses on the local neighborhood of each gene to identify cancer driver genes while correcting for node degree bias [1]. A few studies have explored supervised approaches, incorporating existing knowledge of known driver genes during prediction. Gumpinger et al. used Cancer Gene Census (CGC) annotations to train classifiers for discovering new cancer driver genes [3]. This method models the task as a node-level supervised classification problem in a molecular interaction network, combining network topology with mutation-score features. It achieved more than a 10% improvement in area under the precision–recall curve (AUPRC) over baseline methods.

Network-based approaches derive their strength from integrating weak mutation signals across interconnected genes rather than relying on strong evidence from individual genes. This demonstrates the value of graph-structured representations as a paradigm for cancer driver gene discovery. However, as biological networks continue to grow in size and complexity, classical methods face increasing challenges in scalability and representational efficiency. This motivates the exploration of quantum computing as an alternative framework for representing and processing such structured data, including PPI networks. In particular, quantum machine learning (QML), has been proposed as a tool with potential for inference tasks [15–18]. Recent work has begun applying quantum computing to biological network inference problems, including link prediction tasks [19–21], demonstrating potential speedups over classical baselines, further supporting the potential of quantum approaches in this domain.

Classical methods often rely on local statistical descriptors, such as neighborhood means or low-order moments, to capture structural information in graphs [3, 22]. However, quantum embedding strategies that incorporate such statistics remain unexplored. Moreover, many existing QML approaches neglect the gate complexity of state preparation, limiting their practicality as end-to-end quantum solutions [23, 24]. Motivated by these gaps, we introduce a quantum protocol that embeds low-order statistical moments of features in the one-hop and two-hop neighborhoods into amplitude-encoded quantum states.

In this work, we introduce the *Quantum Multi-order Moment Embedding* (QMME) protocol, which encodes local graph statistics into quantum states for inference. We apply this feature map to a PPI network, where nodes represent genes and features correspond to MutSig P-values. Thus, we design an almost fully quantum pipeline for node-level binary classification of genes as cancer drivers. Classification is performed using the swap-test classifier [25], a non-parametric, distance-based method that employs a quantum kernel based on state similarity.

The entire process, from embeddings to inference, is carried out within the quantum domain, requiring no classical optimization. It relies only on minimal data access assumptions, enabling straightforward circuit design, and involves only limited classical post-processing, inherent to the swap-test classifier. The pipeline explicitly addresses state preparation with practical circuit construction, ensuring embeddings can be constructed with logarithmic qubit resources and polynomial-depth circuits. We validate our approach through numerical simulations on real graphs, evaluating classification performance alongside quantum resource metrics such as circuit depth and query complexity. This work establishes a proof-of-principle for end-to-end quantum learning on graph-structured biological data and lays groundwork for future quantum models in complex network inference tasks.

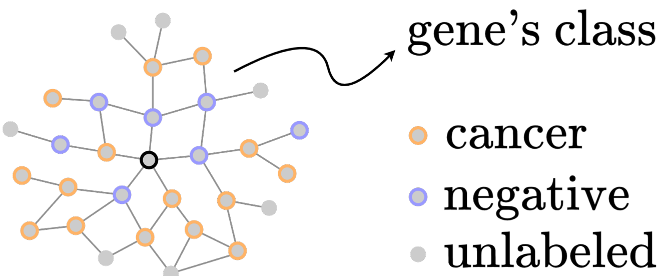

FIG. 1. **Prediction of cancer driver genes.** A PPI network where nodes (genes) are colored by class. The aim is to classify genes and identify cancer drivers based on network structure.

II. PROBLEM SETUP

A. Notation

Protein-protein interaction (PPI) networks model interactions between genes as nodes connected by edges as in fig. 1. Each gene's association with cancer can be quantified using the MutSig P-value [13, 14], which reflects the statistical significance of mutations considering mutational burden, clustering, and functional impact [3]. Formally, we represent a PPI network as an unweighted, undirected graph $\mathcal{G} = (V, E, \boldsymbol{x})$ with $|V| = N$ nodes, where $V = [N]$ denotes the set of node indices and $E \subseteq V \times V$ is the edge set. The connectivity of $\mathcal{G}$ is described by its adjacency matrix $A \in \{0,1\}^{N\times N}$, where $A_{vu} = 1$ indicates the presence of an edge between nodes v and u, and $A_{vu} = 0$ its absence. Define the immediate neighborhood of v as

$$\mathcal{N}_v^1 = \{u \in V \mid A_{vu} = 1\}. \tag{1}$$

We assume $\mathcal{N}_v^1$ is ordered by increasing node index. This allows us to define the function $r(v, \ell)$ as

$$r(v, \ell) := \mathcal{N}_v^1(\ell), \quad \ell \in \{0, \dots, |\mathcal{N}_v^1| - 1\}, \tag{2}$$

which returns the index of the ℓ-th neighbor of v in $\mathcal{N}_v^1$. Equivalently, $r(v, \ell)$ is the column index of the ℓ-th nonzero entry in row v of A. If $\ell \geq |\mathcal{N}_v^1|$, i.e., the function returns a flag value indicating no such neighbor exists. The two-hop neighborhood $\mathcal{N}_v^2 \subseteq V$ is defined as

$$\mathcal{N}_v^2 = \{w \in V \mid \exists u \in V,\, A_{vu} = 1 \wedge A_{uw} = 1\}. \tag{3}$$

Each node $v \in V$ has a scalar feature value $x_v \in \mathbb{R}$, representing its MutSig P-value quantifying the gene's mutation significance. The collection of all node features is denoted by the vector

$$\boldsymbol{x} = (x_1, x_2, \dots, x_N) \in \mathbb{R}^N. \tag{4}$$

B. Node Embeddings

The low-order origin (or raw) moments of a scalar random variable $X \sim \mathcal{P}$ are defined as $\mathbb{E}[X^i]$, where i denotes the order of the moment [26]. These moments are uncentered (i.e., calculated about the origin) and capture fundamental properties of the distribution. The first four are the mean $\mathbb{E}[X]$, the uncentered quadratic $\mathbb{E}[X^2]$, cubic $\mathbb{E}[X^3]$, and quartic $\mathbb{E}[X^4]$ moments. Low-order origin moments serve as the building blocks for the more familiar summary statistics of variance, skewness, and kurtosis, which are centered or standardized variants derived from them. Formally, we define the *origin moment embedding* as the mapping

$$\overline{\varphi}(X) = \big[\mathbb{E}[X],\ \ \mathbb{E}[X^2],\ \ \mathbb{E}[X^3],\ \ \mathbb{E}[X^4]\big], \tag{5}$$

which collects the first four origin moments of X. Since the true distribution $\mathcal{P}$ is generally unknown, these moments are typically estimated empirically. For a finite sample $\boldsymbol{s} = [s_1, \dots, s_k]$ of X, with $s_j \in \mathbb{R}$, the corresponding *sample raw moment embedding* is given by

$$\varphi(\boldsymbol{s}) = \left[\sum_{j=1}^{k} \frac{s_j}{k},\quad \sum_{j=1}^{k} \frac{s_j^2}{k},\quad \sum_{j=1}^{k} \frac{s_j^3}{k},\quad \sum_{j=1}^{k} \frac{s_j^4}{k}\right]. \tag{6}$$

which returns the first four sample origin moments from the sample, thus, describing its distributional shape. The hop order $c \in \{1,2\}$ specifies the neighborhood distance from node v. Given $\boldsymbol{x}$ defined in eq. (4),

$$\mathcal{X}_v^c = \{x_u \in \boldsymbol{x} \mid u \in \mathcal{N}_v^c\}, \tag{7}$$

denotes the set of feature values of nodes at hop-distance c from v, which are samples from a local distribution $\mathcal{P}_v^c$. The function $\varphi(\cdot)$ is applied to each $\mathcal{X}_v^c$, such that

$$\boldsymbol{m}_v^c := \varphi(\mathcal{X}_v^c) \in \mathbb{R}^4. \tag{8}$$

For vertex $v \in V$, the node embedding is then the concatenation of $\boldsymbol{m}_v^c$ from immediate and two-hop neighborhoods:

$$\boldsymbol{m}_v = \left[\boldsymbol{m}_v^1, \boldsymbol{m}_v^2\right] \in \mathbb{R}^8, \tag{9}$$

defining the *node embedding function* $\boldsymbol{m}_v : V \to \mathbb{R}^8$.

C. Problem Statement

We consider a supervised binary classification task on a graph $\mathcal{G} = (V, E, \boldsymbol{x})$, where a subset of nodes $V_l \subseteq V$ is labeled. For these nodes, $y_v = 1$ for $v \in V_1 \subset V_l$ and $y_v = 0$ for $v \in V_0 \subset V_l$, corresponding to the positive and negative classes, respectively. This is a node-level classification task, i.e., the aim is to predict labels for individual nodes. Our goal is to predict the label $\hat{y}_v$ for any node $v \in V$, using a method that combines a *quantum node embedding* unitary operator $U_\mathcal{G}$, derived from the node features $\boldsymbol{x}$ and the connectivity E of $\mathcal{G}$, with a quantum binary classifier $\mathcal{C}$. This combination is used to predict whether a node v belongs to the positive or negative class. That is,

$$\mathcal{C}\left(U_\mathcal{G} \left|v\right\rangle \left|0\right\rangle^{\otimes \mathfrak{m}}\right) = \hat{y}_v, \tag{10}$$

where $\hat{y}_v \in \{0,1\}$ denotes the predicted class label for node v, and $\mathfrak{m}$ is the number of data qubits.

The objective is to explicitly define $U_\mathcal{G}$ as the state preparation step and design the full quantum circuit for the supervised classification of cancer driver genes. The input is a PPI network where the node features $\boldsymbol{x}$ are per-gene MutSig P-values. For the labeled data, a set of positively labeled genes V_1 is obtained from CGC data, so the positive class corresponds to cancer driver genes.

III. QUANTUM MULTI-ORDER MOMENT EMBEDDING (QMME)

A. Input Model

We base our algorithm on having query access to the graph $\mathcal{G} = (V, E, \boldsymbol{x})$ through the adjacency list model for bounded-degree graphs [27–29]. In this input model, the graph has N vertices and bounded degree $s = 2^{\mathfrak{s}}$, i.e., at most s nonzero entries in each row of the adjacency matrix [30, 31]. To accommodate an attributed network, we extend this model by adding a feature query. Thus, the three available query operations are the following:

1. **neighbor query:** given $v \in V$ and $\ell \in \mathbb{Z}$, returns the ℓ-th neighbour of v if $\ell \leq k_v$, and $*$ otherwise,

2. **feature query:** given $v \in V$, returns the value x_v,

3. **degree query:** given $v \in V$, returns the degree k_v.

A quantum equivalent of this input model can be described by defining three corresponding unitary operators O_L, $\tilde{O}_X$, and $\tilde{O}_K$, acting on appropriate Hilbert spaces. Here, O_L provides access to a node's immediate neighborhood, $\tilde{O}_X$ encodes node features, and $\tilde{O}_K$ allows access to the number of c-hop neighbors. The so-called *oracles* act as:

$$O_L : \left|v\right\rangle \left|\ell\right\rangle \mapsto \left|v\right\rangle \left|r(v,\ell)\right\rangle, \tag{11}$$

$$\tilde{O}_X : \left|v\right\rangle \left|0^{d'}\right\rangle \mapsto \left|v\right\rangle \left|\tilde{x}_v\right\rangle, \tag{12}$$

$$\tilde{O}_K : \left|v,c\right\rangle \left|0^{d'}\right\rangle \mapsto \left|v,c\right\rangle \left|\tilde{k}_v^c\right\rangle, \tag{13}$$

where v is the node index, ℓ the neighbor index, $\tilde{x}_v$ the d'-bit fixed point representation of the node feature value $x_v \in \boldsymbol{x}$, and $\tilde{k}_v^c$ the binary representation of the node degree $k_v^c = |\mathcal{N}_v^c|$, i.e., the number of c-hop neighbors.

Although oracle implementation on quantum hardware remains an active topic of research [32], our work assumes coherent access, allowing queries to be made in superposition, which is a standard assumption in the theoretical framework of quantum algorithms. We also assume access to the controlled versions of the oracles O_L, $\tilde{O}_X$, and $\tilde{O}_K$ and their inverses. Since these oracles are unitary and represented by real-valued matrices, their inverses reduce to their transposes. Neighbor and feature queries, whether classical or quantum, are counted as $\mathcal{O}(1)$ in query complexity. The degree query can be emulated by $\mathcal{O}(\log s)$ neighbor queries via binary search [28]. We introduce $\tilde{O}_K$ as a generalization of the degree query, enabling access to the size of a c-hop neighborhood, with $c \in \{1,2\}$. Thus, $\tilde{O}_K$ can be constructed from O_L with $\mathcal{O}\left(\log^2 s\right)$ queries.

B. Rotation Operators

The goal of the embedding circuit is to encode local graph statistics into the amplitudes of quantum states. This is done using controlled single-qubit rotations, where the rotation angles are determined by values encoded in the computational basis and retrieved from the oracles. To this end, we define two rotation operators, F_X and $F_{K^{-1}}$. Given a node feature $x_v \in \mathbb{R}$ with $|x_v| \leq 1$, and its binary encoding $\tilde{x}_v$, the *feature rotation operator* F_X [30]

is defined as

$$F_X : |0\rangle |\tilde{x}_v\rangle \mapsto \left(x_v |0\rangle + \sqrt{1 - |x_v|^2}\, |1\rangle \right) |\tilde{x}_v\rangle , \tag{14}$$

which corresponds to a single-qubit rotation $R_y(\theta)$ with angle $\theta = 2\arcsin(x_v)$. This encodes the scalar value x_v into the amplitude of the $|0\rangle$ component. Similarly, we define the *degree-inverse rotation operator* $F_{K^{-1}}$ [33] as

$$F_{K^{-1}} : |0\rangle |\tilde{k}_v^c\rangle \mapsto \left(\frac{1}{k_v^c} |0\rangle + \sqrt{1 - \left|\frac{1}{k_v^c}\right|^2}\, |1\rangle \right) |\tilde{k}_v^c\rangle . \tag{15}$$

These operators are implemented using multi-controlled R_y rotation gates [30], where the control register holds the binary input $\tilde{x}_v$ or $\tilde{k}_v^c$. The rotation angles are $\theta(\tilde{x}_v) = 2\arcsin(x_v)$ and $\theta(\tilde{k}_v^c) = 2\arcsin(1/k_v^c)$, respectively.

C. Feature and Degree-Inverse Oracles

To amplitude-encode the node embedding $\boldsymbol{m}_v$ into quantum states, we introduce two derived oracles O_X and O_K^{-1}, which use the basis-encoding input oracles $\tilde{O}_X$ and $\tilde{O}_{K^{-1}}$, along with rotation operators F_X and F_K^{-1}. The *feature oracle* O_X encodes powers of the feature value x_v conditioned on the control register $|i\rangle$. Specifically, the amplitude of the component where the ancilla register is in state $|0\rangle^{\otimes 4}$ is mapped to x_v^i, which relates to the i-th order term in the raw moment embedding function $\varphi(\cdot)$ from eq. (6). The oracle O_X acts on a control register, where the integer i in the control register $|i\rangle$ dictates how many of the ancilla qubits are rotated by F_X, as follows:

$$\begin{aligned} |v,i\rangle |0\rangle^{\otimes(4+d')} &\xrightarrow{\tilde{O}_X} |v,i\rangle |0\rangle |\tilde{x}_v\rangle \\ &\xrightarrow{C\text{-}F_X} |v,i\rangle \left(x_v |0\rangle + \sqrt{1-|x_v|^2}\, |1\rangle \right)^{\otimes i} |0\rangle^{\otimes(4-i)} |\tilde{x}_v\rangle \\ &\xrightarrow{\tilde{O}_X^T} |v,i\rangle \left(x_v |0\rangle + \sqrt{1-|x_v|^2}\, |1\rangle \right)^{\otimes i} |0\rangle^{\otimes(4-i+d')} . \end{aligned} \tag{16}$$

Omitting the intermediate register $|\tilde{x}_v\rangle$, fig. 2 summarizes the circuit of oracle O_X, which can be expressed as

$$O_X : |v,i\rangle |0\rangle^{\otimes 4} \mapsto |v,i\rangle \left(x_v |0\rangle + \sqrt{1-|x_v|^2}\, |1\rangle \right)^{\otimes i} |0\rangle^{\otimes(4-i)} . \tag{17}$$

The degree-inverse oracle $O_{K^{-1}}$ amplitude-encodes the inverse degree scaling factors $1/k_v^c$, required to emulate the average present in the moment embedding function $\varphi(\cdot)$, included in the node embedding $\boldsymbol{m}_v$. It performs a controlled rotation that encodes the value $1/k_v^c$ as the amplitude of the $|0\rangle$ state of an ancilla qubit, as follows:

$$\begin{aligned} |v,c\rangle |0\rangle^{\otimes(1+d')} &\xrightarrow{\tilde{O}_K} |v,c\rangle |0\rangle |\tilde{k}_v\rangle \\ &\xrightarrow{F_{K^{-1}}} |v,c\rangle \left(\frac{1}{k_v^c} |0\rangle + \sqrt{1 - \left|\frac{1}{k_v^c}\right|^2}\, |1\rangle \right) |\tilde{k}_v\rangle \\ &\xrightarrow{\tilde{O}_K^T} |v,c\rangle \left(\frac{1}{k_v^c} |0\rangle + \sqrt{1 - \left|\frac{1}{k_v^c}\right|^2}\, |1\rangle \right) |0\rangle^{\otimes d'} . \end{aligned} \tag{18}$$

As shown in fig. 3, the intermediate register $|\tilde{k}_v\rangle$ is omitted, and the oracle is represented as:

$$O_{K^{-1}} : |v,c\rangle |0\rangle \mapsto |v,c\rangle \left(\frac{1}{k_v^c} |0\rangle + \sqrt{1 - \left|\frac{1}{k_v^c}\right|^2}\, |1\rangle \right) . \tag{19}$$

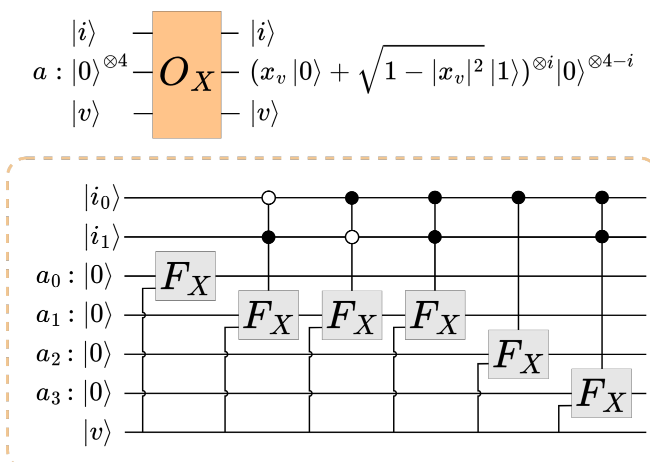

FIG. 2. **Circuit of the feature oracle O_X, based on controlled versions of the rotation operator F_X.** A two-qubit control register $|i\rangle$ selects how many of the four ancilla qubits undergo the rotation F_X: for $i = 00$, only the first qubit; for $i = 01$, the first two; for $i = 10$, the first three; and for $i = 11$, all four. A d'-qubit register is used internally but omitted for clarity.

Together with the neighbor oracle O_L and Hadamard gates, the oracles O_X and $O_{K^{-1}}$ form the core building blocks of the QMME circuit, as depicted in Figure 3.

D. QMME Algorithm

We define the QMME unitary operator $U_{\mathcal{G}}$ as

$$U_{\mathcal{G}} : |v\rangle |0\rangle^{\otimes \mathfrak{m}} \mapsto |v\rangle |\boldsymbol{\psi}_v\rangle , \tag{20}$$

and refer to this mapping as the *quantum node embedding*. The second register, consisting of $\mathfrak{m}$ qubits, is called the *data register*. The total number of qubits is

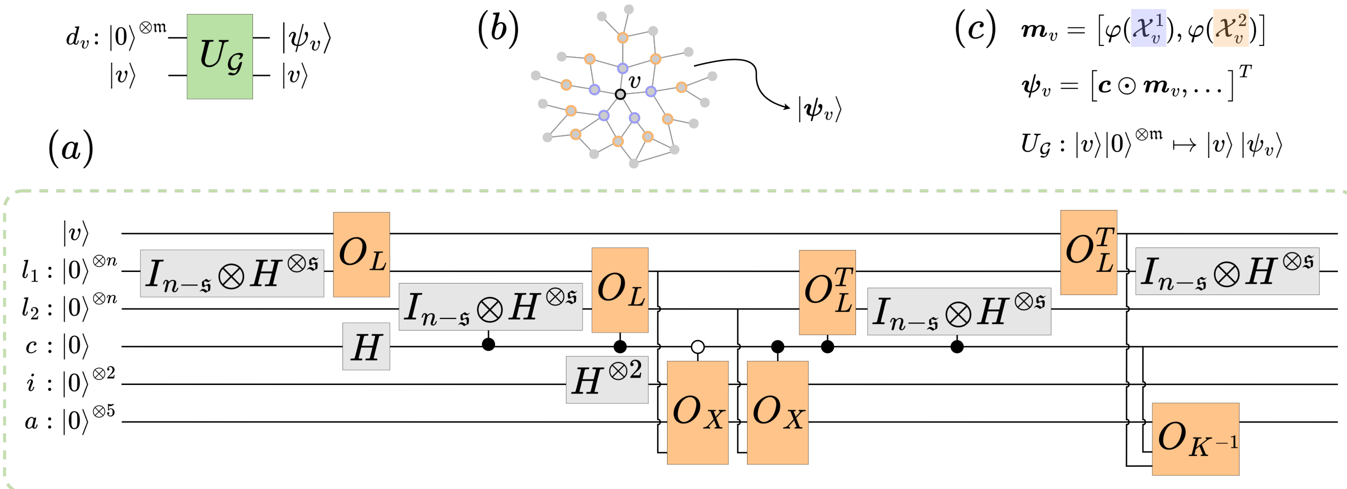

FIG. 3. **Quantum Multi-order Moment Embedding (QMME) protocol.** **(a)** Circuit-level implementation of the unitary operator $U_{\mathcal{G}} : |v\rangle\,|0\rangle^{\otimes\mathfrak{m}} \mapsto |v\rangle\,|\boldsymbol{\psi}_v\rangle$, which amplitude-encodes low-order statistical moments of node v's neighborhood, shown in (b). Here, $|v\rangle$ encodes the node index $v \in V$, and d_v is the $\mathfrak{m}$-qubit data register initialized to zero with $\mathfrak{m} = \mathcal{O}(\log N)$, composed of registers for immediate and second-hop neighbors (ℓ_1, ℓ_2), a control qubit c, a two-qubit register i indexing the i-th moment, and a five-qubit ancilla register a. The circuit accesses the network in (b) via the quantum oracles in eq. (13). **(b)** Input PPI network $\mathcal{G} = (V, E, \boldsymbol{x})$ with node features $\boldsymbol{x}$ representing mutation scores. Immediate neighbors of v are shown in purple, second-hop in orange. These provide the data for the embedding $|\boldsymbol{\psi}_v\rangle \in \mathbb{R}^{2^{\mathfrak{m}}}$. **(c)** The node embedding function $\boldsymbol{m}_v$, based on the moments $\varphi(\cdot)$ over v's neighborhood, is encoded as the quantum state $|\boldsymbol{\psi}_v\rangle$.

$\mathfrak{m} = 2\log N + 8$ (excluding work registers), and generally scales as $\mathcal{O}(\log N)$. A schematic of the operator $U_{\mathcal{G}}$ circuit is shown in Figure 3, and step-by-step derivation of the quantum node embedding in eq. (20) is presented in the Appendix. The resulting state $|\boldsymbol{\psi}_v\rangle$ consists of two components:

$$|\boldsymbol{\psi}_v\rangle = |\boldsymbol{\psi}_m\rangle + |\boldsymbol{\psi}_\perp\rangle\,, \tag{21}$$

where $|\boldsymbol{\psi}_m\rangle$ is the *neighborhood-moment component*, capturing the quantum analogue of the classical embedding $\boldsymbol{m}_v \in \mathbb{R}^8$, and $|\boldsymbol{\psi}_\perp\rangle$ is an orthogonal component.

According to the circuit in Figure 3, $|\boldsymbol{\psi}_m\rangle$ is defined as

$$\begin{aligned}
|\boldsymbol{\psi}_m\rangle &= |\boldsymbol{c} \odot \boldsymbol{m}_v\rangle\,|0\rangle^{\otimes(\mathfrak{m}-3)} && (22)\\
&= \sum_{c\in[2]} \frac{1}{2^{3/2}\,s^c} \sum_{i\in[4]} \varphi_i(\mathcal{X}_v^c)\,|c,i\rangle\,|0\rangle^{\otimes(\mathfrak{m}-3)} && (23)\\
&= \sum_{j\in[8]} c_j \cdot m_{v,j}\,|j\rangle\,|0\rangle^{\otimes(\mathfrak{m}-3)}\,, && (24)
\end{aligned}$$

where $\odot$ denotes the element-wise (Hadamard) product, $\varphi_i(\mathcal{X}_v^c)$ are the ith-order moment of the neighborhood $\mathcal{X}_v^c$, and the constant vector $\boldsymbol{c} \in \mathbb{R}^8$ is

$$\boldsymbol{c} = \frac{1}{2^{3/2}}\left[\frac{1}{s}, \frac{1}{s}, \frac{1}{s}, \frac{1}{s}, \frac{1}{s^2}, \frac{1}{s^2}, \frac{1}{s^2}, \frac{1}{s^2}\right]. \tag{25}$$

The orthogonal component $|\boldsymbol{\psi}_\perp\rangle$ of the resulting state is

$$|\boldsymbol{\psi}_\perp\rangle = \sum_{a\neq 0} |\boldsymbol{\psi}_{\perp,a}\rangle\,|a\rangle\,, \tag{26}$$

where $|a\rangle$ denotes computational basis states in the $(\mathfrak{m}-3)$ data qubits, and $|\boldsymbol{\psi}_{\perp,a}\rangle \in \mathbb{R}^{2^{(\mathfrak{m}-3)}}$.

As an example, take the mean of the mutation scores of the immediate neighbors of gene v, i.e., $m_{v,1} = \varphi_1(\mathcal{X}_v^1)$. As in eq. (23), the amplitude of the state $|0_c, 0_i, 0^{(\mathfrak{m}-3)}\rangle$ will be $\frac{\varphi_1(\mathcal{X}_v^1)}{2^{3/2}s}$. As shown in fig. 3, this arises by:

1. preparing an equal superposition over register ℓ_1;
2. O_L retrieves the immediate neighbors' indices;
3. operator O_X encodes the neighbors' feature values into the $|0\rangle$ component of the first ancilla qubit;
4. uncomputing the basis register ℓ_1 via O_L^T leaves the $|0\rangle^{\otimes n}$ component amplitude proportional to the sum of neighbors' features, while others form interference patterns with alternating signs;
5. finally, the ancilla rotation $O_{K^{-1}}$ normalizes by the degree k_v^c, resulting in the mean of $\mathcal{X}_v^c$ encoded in the amplitude of the rotated ancillas at zero.

This procedure describes the action of the unitary $U_{\mathcal{G}}$ for $c, i = 0$. It generalizes to $c\in[2], i\in[4]$, covering immediate and second neighbors, and more moments, yielding

$$\langle v, c, i|\,\langle 0|^{\otimes(\mathfrak{m}-3)}\,U_{\mathcal{G}}\,|v\rangle\,|0\rangle^{\otimes\mathfrak{m}} = \frac{1}{2^{3/2}s^c} \sum_{x_u\in\mathcal{X}_v^c} \frac{x_u^i}{k_v^c} = \frac{\varphi_i(\mathcal{X}_v^c)}{2^{3/2}s^c}. \tag{27}$$

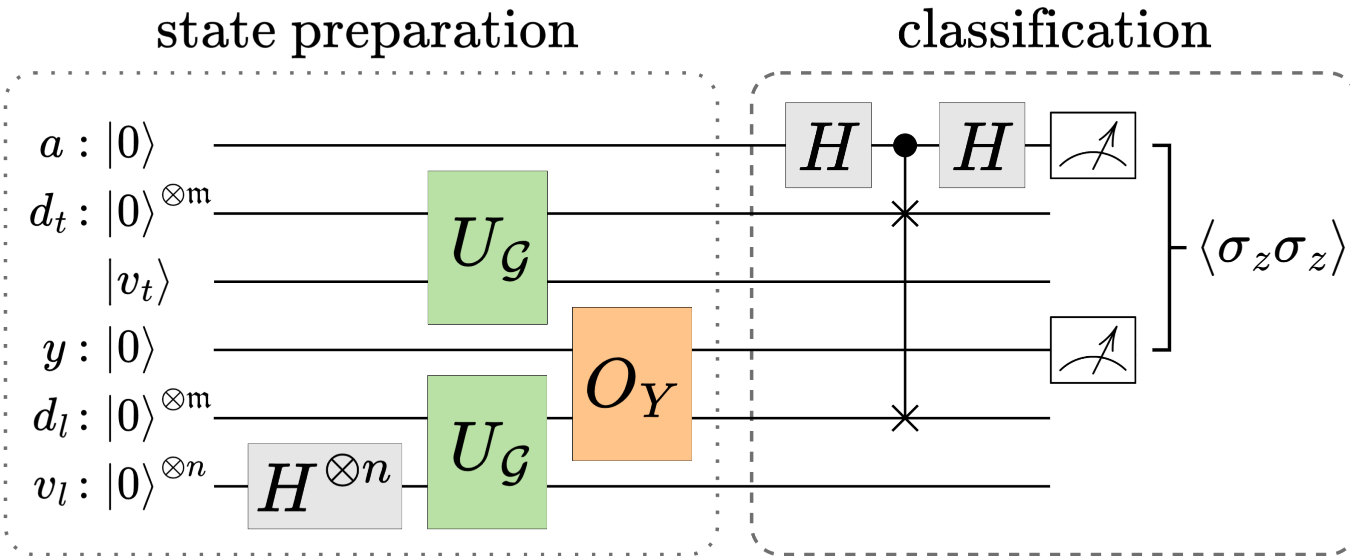

FIG. 4. **Overview of the full pipeline with the QMME protocol for node classification.** The swap-test classifier on top of QMME state preparation. The first register corresponds to the ancilla qubit a, the second the test data qubits d_t, the third the index qubits of the test node v_t, the fourth stores the label qubit y, the fifth the labeled data qubits d_l, and the sixth the index qubits of the labeled nodes v_l. The operator $U_{\mathcal{G}}$, together with the Hadamard gate and the O_Y oracle, are applied to prepare the input quantum state necessary for the classification procedure. The state is then processed by a quantum classifier $\mathcal{C}$ to predict node labels $\hat{y}_v \in \{0,1\}$: The classification outcome is extracted via a swap-test and two-qubit measurement statistics on the ancilla and label registers.

IV. QUANTUM CLASSIFICATION FRAMEWORK

The quantum node embedding operator $U_{\mathcal{G}}$ maps each node in a feature-labeled network to its statistical moments, amplitude-encoded within a quantum state $|\boldsymbol{\psi}_v\rangle$. Given a protein-protein interaction (PPI) network as input, the QMME protocol encodes local statistical moments of the MutSig P-values. Depicted in Figure 4, the quantum classification framework assumes query access to the class labels $y_{v_l} \in \{0,1\}$ for each labeled node $v_l \in V_l$:

1. **label query:** given $v_l \in V_l$, returns the label y_{v_l},

with a quantum equivalent defined by the operator O_Y:

$$O_Y : |v_l\rangle\,|0\rangle \mapsto |v_l\rangle\,|y_{v_l}\rangle\,, \tag{28}$$

encoding the class label $y_{v_l} \in \{0,1\}$ in a single qubit. The proposed quantum classification framework proceeds as follows: after preparing the equal superposition over labeled nodes, the embedding unitary $U_{\mathcal{G}}$ is applied both to this register v_l and to the test node register v_t. The class labels are loaded onto the label qubit via O_Y. Classification follows via the swap-test classifier [25], where the measurement statistics determine the final class prediction of the test node. The full pipeline may be described as a sequence of quantum state evolutions:

$$\begin{aligned}
&|0\rangle\,|0\rangle^{\otimes\mathfrak{m}}\,|v_t\rangle\,|0\rangle\,|0\rangle^{\otimes\mathfrak{m}}\,|0\rangle^{\otimes n}\\
&\xrightarrow{H_{v_l}^{\otimes n}} \frac{1}{\sqrt{|V_l|}}\sum_{v_l\in V_l}|0\rangle\,|0\rangle^{\otimes\mathfrak{m}}\,|v_t\rangle\,|0\rangle\,|0\rangle^{\otimes\mathfrak{m}}\,|v_l\rangle\\
&\xrightarrow{U_{\mathcal{G}}} \frac{1}{\sqrt{|V_l|}}\sum_{v_l\in V_l}|0\rangle\,|\boldsymbol{\psi}_t\rangle\,|v_t\rangle\,|0\rangle\,|\boldsymbol{\psi}_l\rangle\,|v_l\rangle\\
&\xrightarrow{O_Y} \frac{1}{\sqrt{|V_l|}}\sum_{v_l\in V_l}|0\rangle\,|\boldsymbol{\psi}_t\rangle\,|v_t\rangle\,|y_l\rangle\,|\boldsymbol{\psi}_l\rangle\,|v_l\rangle\\
&\xrightarrow{H_a\cdot\text{c-swap}\cdot H_a} |\boldsymbol{\psi}_f\rangle = \frac{1}{\sqrt{|V_l|}}\sum_{v_l\in V_l}\left(|0\rangle\,|\boldsymbol{\psi}_+\rangle + |1\rangle\,|\boldsymbol{\psi}_-\rangle\right)|v_l\rangle\,,
\end{aligned} \tag{29}$$

where $|\boldsymbol{\psi}_\pm\rangle = |\boldsymbol{\psi}_t\rangle\,(|v_t\rangle\,|y_l\rangle)\,|\boldsymbol{\psi}_l\rangle \pm |\boldsymbol{\psi}_l\rangle\,(|v_t\rangle\,|y_l\rangle)\,|\boldsymbol{\psi}_t\rangle$. The swap-test classifier computes the fidelity, i.e., the squared overlap between two states. This yields $|\langle\psi|\phi\rangle|^2$ for two pure states $|\psi\rangle$ and $|\phi\rangle$, and more generally $\mathrm{Tr}(\rho\sigma)$ for two possibly mixed states ρ and σ. Fidelity serves as a similarity measure (one if the states are identical, zero if they are orthogonal), making it a natural distance measure in the quantum feature space. Classification thus relies directly on distances between training and test data, unifying feature mapping and classification into a single quantum procedure without requiring an intermediate classical support vector machine.

The measurement and post-processing involves evaluating the expectation value of the two-qubit observable $\sigma_z^{(a)}\sigma_z^{(l)}$, where the superscripts indicate the ancilla and label qubit. For the final state $|\boldsymbol{\psi}_f\rangle$, $\langle\sigma_z^{(a)}\sigma_z^{(l)}\rangle$ is given by

$$\langle\sigma_z^{(a)}\sigma_z^{(l)}\rangle = \mathrm{Tr}\left(\sigma_z^{(a)}\sigma_z^{(l)}\,|\boldsymbol{\psi}_f\rangle\langle\boldsymbol{\psi}_f|\right) \tag{30}$$

$$= \frac{1}{|V_l|}\sum_{v_l\in V_l}(-1)^{y_{v_l}}|\langle\boldsymbol{\psi}_t|\boldsymbol{\psi}_{v_l}\rangle|^2. \tag{31}$$

In practice, these statistics are obtained by repeating r times the two-qubit projective measurement in the σ_z basis. Using $\mathrm{Tr}(\sigma_z\,|y_{v_l}\rangle\langle y_{v_l}|) = 1$ for $y_{v_l} = 0$ and -1 for $y_{v_l} = 1$, the expectation value is calculated as

$$\langle\sigma_z^{(a)}\sigma_z^{(l)}\rangle = \frac{1}{r}\left(c_{00} - c_{01} - c_{10} + c_{11}\right), \tag{32}$$

where c_{al} counts measurements with the ancilla in state a and the label qubit in state l. The test data are classified as 0 if $\langle\sigma_z^{(a)}\sigma_z^{(l)}\rangle > 0$ and 1 if negative, i.e.,

$$\hat{y}_v = \frac{1}{2}\left(1 - \mathrm{sgn}\,\langle\sigma_z^{(a)}\sigma_z^{(l)}\rangle\right) \tag{33}$$

$$= \mathcal{C}\left(U_{\mathcal{G}}\,|v\rangle\,|0\rangle^{\otimes\mathfrak{m}}\right). \tag{34}$$

which resembles the earlier formulation in Equation (10). Here, $\mathcal{C}$ is the swap-test classifier and $U_{\mathcal{G}}$ corresponds to the Quantum Multi-order Moment Embedding (QMME), encoding local properties in the network $\mathcal{G}$.

V. NUMERICAL SIMULATIONS

Classical simulations of quantum circuits. [ongoing] performance against state-of-the-art classical baselines. The implementation and all data used in this work are openly available on GitHub.

A. Dataset Description

B. Results

Finally, to validate our code, we replicated the simplest swap-test classifier example from the reference paper, involving two nodes each with two features. This test confirmed the correctness of our swap-test classifier implementation and helped isolate errors unrelated to core classification logic.

VI. COMPLEXITY ANALYSIS

We analyze the complexity of the QMME circuit in terms of gate count, circuit depth, and required qubits. Let $|V_l| = 2^n$ be the number of labeled nodes and d' the precision (in bits) of the encoded features and degrees. Loading a feature vector via $\tilde{O}_X$ maps $|v\rangle\,|0\rangle \mapsto |v\rangle\,|\tilde{x}_v\rangle$ and can be implemented with $\mathcal{O}$(did not find) gates using structured memory access or qRAM. The rotation F_X applies a controlled $R_y(2\arccos x_v)$ gate, with θ computed on-the-fly via quantum arithmetic, contributing $\mathcal{O}(\text{poly } n + d')$ gates. The oracle $\tilde{O}_{K^{-1}}$ and $F_{K^{-1}}$ follow the same complexity. The total number of oracle calls per QMME state preparation is constant, with two calls to each oracle (and its inverse), plus up to five amplitude-encoding operations. The number of Hadamard gates scales as $\mathcal{O}(n + \mathfrak{s})$, and the total gate count is dominated by arithmetic and oracle logic. Work qubits include l for input registers, $\mathfrak{s}$ for neighbors basis-encoding, and $\mathcal{O}(1)$ for control. The total number of qubits is thus $\mathcal{O}(n+l+\mathfrak{s})$, where d' depends on precision. [More on Hello [34].]

TABLE I. **Quantum circuit QMME complexity.** Estimates for n-sparse and general graphs

Metric	Adjacency Matrix	
	s-sparse	**General**
Gate count	$\mathcal{O}(\log s)$	$\mathcal{O}(\log N)$
Circuit depth	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Qubit count	$\mathcal{O}(\log N + d')$	$\mathcal{O}(\log\log N + d')$
Query complexity	$\mathcal{O}(\log^2 s)$	$\mathcal{O}(\log^2 N)$

VII. DISCUSSION AND CONCLUSIONS

QMMEP realizes a fully quantum node-classification pipeline - no classical optimization loops; all encoding, embedding, and inference occur quantumly. We demonstrate polynomial resource scaling and solid performance on synthetic data. Future work includes extensions to deeper moments, weighted graphs, and implementation on near-term hardware. An architecture that others can build on (generalizable moment-encoding circuit) [22]. Nevertheless, the QMME protocol may be applied to any node-level binary classification on graph-structured data with feature-labeled nodes.

Potential improvements or extensions include multi-class classification (either gene categories or even outside the scope of biological networks), and mention applications to other types of nets. [mention limitations] [adress o porquê de escolher momentos até ordem 4. e de como cada parte do circuito (1-hop, 2-hop) contribui para a performance]

[1] H. Horn, M. S. Lawrence, C. R. Chouinard, Y. Shrestha, J. X. Hu, E. Worstell, E. Shea, N. Ilic, E. Kim, A. Kamburov, *et al.*, Nature methods **15**, 61 (2018).

[2] M. Nourbakhsh, K. Degn, A. Saksager, M. Tiberti, and

E. Papaleo, Briefings in Bioinformatics **25**, bbad519 (2024).

[3] A. C. Gumpinger, K. Lage, H. Horn, and K. Borgwardt, Bioinformatics **36**, i508 (2020).

[4] L. A. Garraway and E. S. Lander, Cell **153**, 17 (2013).

[5] B. Vogelstein, N. Papadopoulos, V. E. Velculescu, S. Zhou, L. A. Diaz Jr, and K. W. Kinzler, science **339**, 1546 (2013).

[6] G. M. Frampton, A. Fichtenholtz, G. A. Otto, K. Wang, S. R. Downing, J. He, M. Schnall-Levin, J. White, E. M. Sanford, P. An, *et al.*, Nature biotechnology **31**, 1023 (2013).

[7] M. A. Reyna, M. D. Leiserson, and B. J. Raphael, Bioinformatics **34**, i972 (2018).

[8] I. A. Kovács, K. Luck, K. Spirohn, Y. Wang, C. Pollis, S. Schlabach, W. Bian, D.-K. Kim, N. Kishore, T. Hao, *et al.*, Nature communications **10**, 1240 (2019).

[9] F. Cheng, I. A. Kovács, and A.-L. Barabási, Nature communications **10**, 1197 (2019).

[10] K. Lage, E. O. Karlberg, Z. M. Størling, P. I. Olason, A. G. Pedersen, O. Rigina, A. M. Hinsby, Z. Tümer, F. Pociot, N. Tommerup, *et al.*, Nature biotechnology **25**, 309 (2007).

[11] T. Li, R. Wernersson, R. B. Hansen, H. Horn, J. Mercer, G. Slodkowicz, C. T. Workman, O. Rigina, K. Rapacki, H. H. Stærfeldt, *et al.*, Nature methods **14**, 61 (2017).

[12] D. Szklarczyk, A. L. Gable, D. Lyon, A. Junge, S. Wyder, J. Huerta-Cepas, M. Simonovic, N. T. Doncheva, J. H. Morris, P. Bork, *et al.*, Nucleic acids research **47**, D607 (2019).

[13] J. G. Lohr, P. Stojanov, M. S. Lawrence, D. Auclair, B. Chapuy, C. Sougnez, P. Cruz-Gordillo, B. Knoechel, Y. W. Asmann, S. L. Slager, *et al.*, Proceedings of the National Academy of Sciences **109**, 3879 (2012).

[14] M. S. Lawrence, P. Stojanov, C. H. Mermel, J. T. Robinson, L. A. Garraway, T. R. Golub, M. Meyerson, S. B. Gabriel, E. S. Lander, and G. Getz, Nature **505**, 495 (2014).

[15] P. Wittek, *Quantum machine learning: what quantum computing means to data mining* (Academic Press, 2014).

[16] M. Schuld, I. Sinayskiy, and F. Petruccione, Contemporary Physics **56**, 172 (2015).

[17] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, Nature **549**, 195 (2017).

[18] D. K. Park, C. Blank, and F. Petruccione, Physics Letters A **384**, 126422 (2020).

[19] J. P. Moutinho, A. Melo, B. Coutinho, I. A. Kovács, and Y. Omar, Physical Review A **107**, 032605 (2023).

[20] D. Magano, J. Moutinho, and B. Coutinho, SciPost Physics Core **6**, 058 (2023).

[21] J. P. Moutinho, D. Magano, and B. Coutinho, Scientific Reports **14**, 1026 (2024).

[22] W. Bi, L. Du, Q. Fu, Y. Wang, S. Han, and D. Zhang, in *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining* (2023) pp. 132–140.

[23] H. Zhao, L. Lewis, I. Kannan, Y. Quek, H.-Y. Huang, and M. C. Caro, PRX Quantum **5**, 040306 (2024).

[24] Y. Wang and J. Liu, Reports on Progress in Physics (2024).

[25] C. Blank, D. K. Park, J.-K. K. Rhee, and F. Petruccione, npj Quantum Information **6**, 41 (2020).

[26] J. Shao, *Mathematical statistics* (Springer, 1999).

[27] O. Goldreich and D. Ron, in *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing* (1997) pp. 406–415.

[28] O. Goldreich, *Introduction to property testing* (Cambridge University Press, 2017).

[29] S. Ben-David, A. M. Childs, A. Gilyén, W. Kretschmer, S. Podder, and D. Wang, SIAM Journal on Computing **53**, FOCS20 (2024).

[30] L. Lin, arXiv preprint arXiv:2201.08309 (2022).

[31] D. Camps, L. Lin, R. Van Beeumen, and C. Yang, SIAM Journal on Matrix Analysis and Applications **45**, 801 (2024).

[32] P. Kuklinski and B. Rempfer, arXiv preprint arXiv:2401.04234 (2024).

[33] Y. Tong, D. An, N. Wiebe, and L. Lin, Physical Review A **104**, 032422 (2021).

[34] M. Möttönen, J. J. Vartiainen, V. Bergholm, and M. M. Salomaa, Physical review letters **93**, 130502 (2004).

[35] S. Arora and B. Barak, *Computational complexity: a modern approach* (Cambridge University Press, 2009).

[36] R. Levi, R. Rubinfeld, and A. Yodpinyanee, in *Proceedings of the 27th ACM symposium on Parallelism in Algorithms and Architectures* (2015) pp. 59–61.

[37] L. Cincio, Y. Subaşı, A. T. Sornborger, and P. J. Coles, New Journal of Physics **20**, 113022 (2018).

[38] J. Biamonte, M. Faccin, and M. De Domenico, Communications Physics **2**, 53 (2019).

[39] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, Nature **567**, 209 (2019).

[40] J. M. Martyn, Z. M. Rossi, A. K. Tan, and I. L. Chuang, PRX quantum **2**, 040203 (2021).

[41] S. Behnezhad, M. Roghani, and A. Rubinstein, in *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)* (IEEE, 2023) pp. 2322–2335.

Quantum Node Embedding Step-by-step Derivation

U_r denotes the operator U acting on register r, with identity on all other registers; C - U_r the controlled application of U_r, with the control qubit in register c being in state $|1\rangle$; and $\mathrm{C}^{(0)}$ - U_r indicates control on the $|0\rangle$ state of c.

$$
\begin{aligned}
&|v, 0^{\mathfrak{m}}\rangle \quad \text{where } \mathfrak{m} = 2\log N + 8, \text{omitting work qubits,} \\
&\xrightarrow{I_{n-s}\otimes H_{l_1}^{\otimes s}} \frac{1}{\sqrt{s}} |v, 0^3\rangle \sum_{\ell_1\in[s]} |\ell_1, 0^{(n+5)}\rangle \\
&\xrightarrow{(O_L)_{v,l_1}} \frac{1}{\sqrt{s}} |v, 0^3\rangle \sum_{\ell_1\in[s]} |r(v,\ell_1), 0^{(n+5)}\rangle \\
&\xrightarrow{H_c} \frac{1}{\sqrt{2s}} |v\rangle \sum_{c\in[2]} \sum_{\ell_1\in[s]} |c, 0^2, r(v,\ell_1), 0^{(n+5)}\rangle \\
&\xrightarrow{I_{n-s}\otimes C\text{-}H_{\ell_2}^{\otimes s}} \frac{1}{\sqrt{2s}} |v\rangle \sum_{\ell_1\in[s]} \left(|0_c, 0^2, r(v,\ell_1), 0^{(n+5)}\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2\in[s]} |1_c, 0^2, r(v,\ell_1), \ell_2, 0^5\rangle \right) \\
&\xrightarrow{C\text{-}(O_L)_{\ell_1,\ell_2}} \frac{1}{\sqrt{2s}} |v\rangle \sum_{\ell_1\in[s]} \left(|0_c, 0^2, r(v,\ell_1), 0^{(n+5)}\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2\in[s]} |1_c, 0^2, r(v,\ell_1), r(\mathcal{N}_v^1(\ell_1),\ell_2), 0^5\rangle \right) \\
&\xrightarrow{H_i^{\otimes 2}} \frac{1}{2^{3/2}\sqrt{s}} |v\rangle \sum_{\ell_1\in[s]} \sum_{i\in[4]} \left(|0_c, i, r(v,\ell_1), 0^{(n+5)}\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2\in[s]} |1_c, i, r(v,\ell_1), r(\mathcal{N}_v^1(\ell_1),\ell_2), 0^5\rangle \right) \\
&\xrightarrow{C^{(0)}\text{-}(O_X)_{i,a,l_1}} \frac{1}{2^{3/2}\sqrt{s}} |v\rangle \sum_{\ell_1,i} \left(\left(x^i_{r(v,\ell_1)} |0_c, i, r(v,\ell_1), 0^{(n+4)}\rangle + |\perp\rangle \right) |0\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2\in[s]} |1_c, i, r(v,\ell_1), r(\mathcal{N}_v^1(\ell_1),\ell_2), 0^5\rangle \right) \\
&\xrightarrow{C\text{-}(O_X)_{i,a,\ell_2}} \frac{|v\rangle}{2^{3/2}\sqrt{s}} \sum_{\ell_1,i} \left(x^i_{r(v,\ell_1)} |0_c, i, r(v,\ell_1), 0^{(n+5)}\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2\in[s]} x^i_{r(\mathcal{N}_v^1(\ell_1),\ell_2)} |1_c, i, r(v,\ell_1), r(\mathcal{N}_v^1(\ell_1),\ell_2), 0^5\rangle \right) + |v\rangle|\perp\rangle|0\rangle \\
&\xrightarrow{C\text{-}(O_L^T)_{\ell_1,\ell_2}} \frac{1}{2^{3/2}\sqrt{s}} |v\rangle \sum_{\ell_1,i} \left(x^i_{r(v,\ell_1)} |0_c, i, r(v,\ell_1), 0^{(n+5)}\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2\in[s]} x^i_{r(\mathcal{N}_v^1(\ell_1),\ell_2)} |1_c, i, r(v,\ell_1), \ell_2, 0^5\rangle \right) + |v\rangle|\perp\rangle|0\rangle \\
&\xrightarrow{I_{n-s}\otimes C\text{-}H_{\ell_2}^{\otimes s}} \frac{1}{2^{3/2}\sqrt{s}} |v\rangle \sum_{\ell_1,i} \left(x^i_{r(v,\ell_1)} |0_c, i, r(v,\ell_1)\rangle + \frac{1}{\sqrt{s}} \sum_{x'\in\mathcal{X}^1_{r(v,\ell_1)}} x'^i |1_c, i, r(v,\ell_1)\rangle \right) |0^{(n+5)}\rangle + |v\rangle|\perp\rangle|0\rangle \\
&\xrightarrow{(O_L^T)_{v,l_1}} \frac{1}{2^{3/2}\sqrt{s}} |v\rangle \sum_{\ell_1,i} \left(x^i_{r(v,\ell_1)} |0_c, i, \ell_1\rangle + \frac{1}{s} \sum_{x'\in\mathcal{X}^1_{r(v,\ell_1)}} x'^i |1_c, i, \ell_1\rangle \right) |0^{(n+5)}\rangle + |v\rangle|\perp\rangle|0\rangle \\
&\xrightarrow{I_{n-s}\otimes H_{l_1}^{\otimes s}} \frac{1}{2^{3/2}s} |v\rangle \sum_{i\in[4]} \left(\sum_{x'\in\mathcal{X}^1_{r(v,\ell_1)}} x'^i |0_c\rangle + \frac{1}{s} \sum_{x''\in\mathcal{X}^2_v} x''^i |1_c\rangle \right) |i, 0^{(2n+5)}\rangle + |v\rangle|\perp\rangle|0\rangle \\
&\xrightarrow{(O_{K^{-1}})_{v,k}} \frac{1}{2^{3/2}s} |v\rangle \sum_{i\in[4]} \left(\sum_{x'\in\mathcal{X}^1_{r(v,\ell_1)}} x'^i |0_c\rangle + \frac{1}{s} \sum_{x''\in\mathcal{X}^2_v} x''^i |1_c\rangle \right) \frac{1}{k_v^c} |i, 0^{(2n+5)}\rangle + |v\rangle|\perp\rangle \\
&= |v\rangle \left(\sum_{i,c} \frac{1}{2^{3/2}s^c} \sum_{x_u\in\mathcal{X}_v^c} \frac{x_u^i}{k_v^c} |c, i, 0^{(2n+5)}\rangle + |\perp\rangle \right) \\
&= |v\rangle \left(\sum_{i,c} \frac{1}{2^{3/2}s^c} \varphi(\mathcal{X}_v^c) |c, i, 0^{(2n+5)}\rangle + |\perp\rangle \right) \\
&= |v\rangle \left(\sum_{j\in[8]} c_j m_{v,j} |j, 0^{(2n+5)}\rangle + |\perp\rangle \right) \quad \text{with } \boldsymbol{c} \text{ defined as in eq. (25).}
\end{aligned}
$$

Complexity Analysis in Detail

TABLE II. **Quantum Gate Complexity and Queries for Various Sources**

Operator	**Description**	**Gate Count**	**Circuit Depth**	**Qubit Count**	**Query Complexity**
H	elementary gate	1	1	1	$\mathcal{O}(1)$
O_L	input model	g_{OL}	d_{OL}	$2n$	$\mathcal{O}(1)$
$\tilde{O}_X$	input model	g_{OX}	d_{OX}	$n + d'$	$\mathcal{O}(1)$
$\tilde{O}_K$	derived from O_L	g_{OK}	d_{OK}	$n + d' + 1$	$\mathcal{O}(\log^2 s)$
F_X	type of R_y	g_{FX}	d_{FX}	$d' + 1$	$\mathcal{O}(F_X)$
$F_{K^{-1}}$	type of R_y (INV)	g_{FX-1}	d_{FK-1}	$d' + 1$	$\mathcal{O}(F_{K^{-1}})$
O_X	$1\,\tilde{O}_X \cdot 5\,\text{c-}F_X \cdot 1\tilde{O}_X^T$	$2g_{OX} + 5g_{FX}$	$2d_{OX} + 5d_{FX}$	$\log N + d' + 5$	$\mathcal{O}(F_X)$
$O_{K^{-1}}$	$1\,\tilde{O}_K \cdot 1\,F_{K^{-1}} \cdot 1\,\tilde{O}_K^T$	$2g_{OK} + g_{F_{K-1}}$	$2d_{OK} + d_{F_{K-1}}$	$\log N + d' + 2$	$\mathcal{O}(\log^2 s + F_{K^{-1}})$

Furthermore, once we have access to the number of elementary gates needed to implement Uf, we obtain immediately the gate complexity of the total algorithm. However, some queries can be (provably) difficult to implement, and then there can be a large gap between the query complexity and gate complexity. In order to obtain a meaningful query complexity analysis, one should also make sure that other components of the quantum algorithm will not end up dominating the total gate complexity, when all factors are taken into account. from scinetici noes.

Gentle intro to quantum circuit: In the standard circuit model of quantum computation, the efficiency of a quantum algorithm is computed in terms of the circuit complexity, the number of basic gates together with the number of qubits used, of the circuits used to implement the algorithm. + Black box algorithms of low query complexity, algorithms that solve a black box problem with few calls to the oracle, are only of practical use if the black box has an efficient implementation. The black box approach is very useful, however, in establishing lower bounds on the circuit complexity of a problem. If the query complexity is (N)—in other words, at least (N) calls to the oracle are required—then the circuit complexity must be at least (N).